

This lab session covers Django Forms (Part 2 – Django form field and file handling). By the end of this session, students should have a very good understanding of how to use Django form fields.

Pre-lab Preparation:

1. The table student and address as in Lab (8).
2. The fake data from mockaroo.com website or any other methods.

[Home](#) [List Books](#) [About](#)

Task 1: One-to-One Students

[+ Add Student \(Task 1\)](#)

Name	Age	Address	Actions (Update/Delete)
amar	22	Riyadh,	Update Delete
salah	40	Qassim ,	Update Delete

Task 2: Many-to-Many Students

[+ Add Student \(Task 2\)](#)

Name	Addresses	Actions (Update/Delete)
Rola	Riyadh	Update Delete
sara	Riyadh, Jeddah	Update Delete

[Go to Gallery \(Task 3\) →](#)

Lab Activities:

Task 1: Create a URL to list, add, update, or delete students and specify addresses (one address for each student) by applying the Django form field with any necessary HTML files and view functions.

```
class Address(models.Model):
```

```
    city = models.CharField(max_length=100)
    street = models.CharField(max_length=100)
    def __str__(self): return f"{self.city}, {self.street}"
```

```
class Student(models.Model):
```

```
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    address = models.OneToOneField(Address, on_delete=models.CASCADE)
```

In view

```
def student_list(request):
    students1 = Student.objects.all()
    students2 = Student2.objects.all()
    return render(request, 'bookmodule/student_list.html', {
        'students1': students1,
        'students2': students2
    })

# --- عمليات Task 1 (One-to-One) ---

def student_add(request):
    if request.method == "POST":
        form = StudentForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('student_list')
    else:
        form = StudentForm()
        return render(request, 'bookmodule/student_form.html', {'form': form, 'title':
'Add Student (Task 1)'})

def student_edit(request, id):
    student = Student.objects.get(id=id)
    if request.method == "POST":
        form = StudentForm(request.POST, instance=student)
        if form.is_valid():
            form.save()
            return redirect('student_list')
    else:
        form = StudentForm(instance=student)
        return render(request, 'bookmodule/student_form.html', {'form': form, 'title':
'Edit Student (Task 1)'})

def student_delete(request, id):
    student = Student.objects.get(id=id)
```

```
student.delete()  
  
return redirect('student_list')
```

Add Student (Task 1)

Name: Age: Address: [Back to List](#)[Home](#) [List Books](#) [About](#)

Task 1: One-to-One Students

[+ Add Student \(Task 1\)](#)

Name	Age	Address	Actions (Update/Delete)
amar	22	Riyadh,	Update Delete
salah	40	Qassim ,	Update Delete
sara	13	Riyadh,	Update Delete

Task 2: Suppose the relation between students and addresses is many-to-many, in which a student can have multiple addresses, and also, one address can be associated with many students. In this task, the code in the previous task should be copied and modified to implement many-to-many relationships.

```
class Address2(models.Model):  
    city = models.CharField(max_length=100)  
    def __str__(self): return self.city  
  
class Student2(models.Model):  
    name = models.CharField(max_length=100)  
    addresses = models.ManyToManyField(Address2)
```

In view

```
def student2_add(request):
    if request.method == "POST":
        form = Student2Form(request.POST)
        if form.is_valid():
            form.save()
            return redirect('student_list')
    else:
        form = Student2Form()
        return render(request, 'bookmodule/student_form.html', {'form': form, 'title':
'Add Student (Task 2)'})

def student2_edit(request, id):
    student = Student2.objects.get(id=id)
    if request.method == "POST":
        form = Student2Form(request.POST, instance=student)
        if form.is_valid():
            form.save()
            return redirect('student_list')
    else:
        form = Student2Form(instance=student)
        return render(request, 'bookmodule/student_form.html', {'form': form, 'title':
'Edit Student (Task 2)'})

def student2_delete(request, id):
    student = Student2.objects.get(id=id)
    student.delete()
    return redirect('student_list')
```

Add Student (Task 2)

Name:

Addresses:

Riyadh

Jeddah

Dammam

Qassim

[Back to List](#)

Task 2: Many-to-Many Students

[+ Add Student \(Task 2\)](#)

Name	Addresses	Actions (Update/Delete)
Rola	Riyadh	Update Delete
sara	Riyadh, Jeddah	Update Delete
amar	Jeddah, Dammam	Update Delete

[Go to Gallery \(Task 3\) →](#)

Note: If needed, student or address models should be copied and renamed to, for example, student2 and address2.

Task 3: Create an additional table (of your own choice) that includes images, and you should build forms to handle data with images through Django.

Note: A few configurations are needed for file handling.

In model:

```
class ProjectGallery(models.Model):
    title = models.CharField(max_length=100)
    image = models.ImageField(upload_to='images/')
```

in view:

```
def gallery_view(request):
    if request.method == "POST":
        form = GalleryForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
```

```
        return redirect('gallery_list')
    else:
        form = GalleryForm()

    images = ProjectGallery.objects.all()
    return render(request, 'bookmodule/gallery.html', {'form': form, 'images':
images})
```

Task 3: Project Gallery

Upload New Image

Title:

Image: لم يتم اختيار ملف

Uploaded Images

web



[Back to Student List](#)